

Proyek Sistem Operasi

Minggu 2: *System Calls* & Manajemen Proses (Bag. 1)

Novalio Daratha
Program Studi Informatika
Universitas Bengkulu

2026

Outline

Apa itu *System Call*?

Definisi

System Call adalah antarmuka (API) antara program aplikasi (*user space*) dengan kernel sistem operasi (*kernel space*).

- Program biasa tidak diizinkan mengakses **hardware** atau memori sistem secara langsung.
- Untuk melakukan operasi sensitif (seperti membaca disk atau membuat proses), program harus meminta bantuan kernel melalui **System Call**.

Hierarki Proses di Linux

Setiap proses di Linux diidentifikasi oleh sebuah PID (*Process ID*).

- Sistem operasi selalu memiliki proses pertama (biasanya `systemd` atau `init` dengan PID 1).
- Proses baru selalu dibuat oleh proses yang sudah ada. Proses yang membuat disebut **Parent Process**, dan yang dibuat disebut **Child Process**.

Melihat Proses

Anda dapat menggunakan perintah `ps -ef` atau `pstree` di terminal untuk melihat struktur **tree** dari proses yang sedang berjalan.

System Call: fork()

fork() digunakan untuk menciptakan *child process* baru.

- *Child process* adalah duplikat persis dari *parent process* (termasuk *memory space*, kode, register).
- Mengembalikan 0 kepada *child*, dan mengembalikan PID *child* kepada *parent*. Mengembalikan -1 jika gagal.

Contoh Penggunaan

```
pid_t pid = fork();
if (pid == 0) {
    // Kode ini hanya dieksekusi oleh child
    printf("Halo dari Child!\n");
} else if (pid > 0) {
    // Kode ini dieksekusi oleh parent
    printf("Halo dari Parent!\n");
}
```

Fungsi Pendukung Proses

`exec()`

Mengganti `*image*` memori dari proses saat ini dengan program baru. Biasanya dipanggil oleh `*child*` setelah `fork()` untuk menjalankan aplikasi lain. (Contoh: `execl`, `execvp`).

`wait()`

Memaksa `*parent process*` untuk berhenti sejenak menunggu `*child process*`-nya selesai dieksekusi. Mencegah terjadinya `*zombie process*`.

`exit()`

Digunakan untuk mengakhiri proses secara normal dan mengembalikan status ke `*parent*`.

Proyek 1: Membuat *Custom Shell*

- **Tujuan:** Mengaplikasikan `fork()`, `exec()`, dan `wait()`.
- **Deskripsi:** Anda akan membuat sebuah program C yang menerima input command dari user (mirip terminal Bash), lalu mengeksekusi perintah tersebut sebagai **child process**.
- **Langkah awal:** Pelajari cara mengambil input dari **user**, **parsing string**, dan membungkus eksekusinya dengan struktur **fork-exec-wait**.

Terima Kasih

Diskusi & Eksplorasi Proyek 1

Novalio Daratha
Program Studi Informatika
Universitas Bengkulu